Metaprograms + Decorators

CS-3180

Metaprogramming

- *Metaprogramming:* programs that treat other programs as data
- Macros: Transform source code at compile time
- Homoiconicity: Code and data share the same representation
- Allows for metaprogramming
- Many other types!
- Focus on decorators today

Decorators

- Decorators transform functions via @
- (Java calls these 'annotations', but lacks first class support like Python)
- "Decorate" a function, modifying its behavior

```
@decorator_name
def my_function(a, b):
    print(f"a={a}, b={b}")
    return a + b
```

Decorator Implementation

- We're are just extending the behavior of the function
- If we were to implement decorators, what information would we need?

Decorator Implementation

- We're are just extending the behavior of the function
- If we were to implement decorators, what information would we need?
- 1. We need to know the original function

Decorator Implementation

- We're are just extending the behavior of the function
- If we were to implement decorators, what information would we need?
- 1. We need to know the original function
- 2. We need to know what modified function was

Decorator Implementation

- We're are just extending the behavior of the function
- If we were to implement decorators, what information would we need?
- 1. We need to know the original function
- 2. We need to know what modified function was
- This feels like a higher order function

Decorator Implementation

- Let's derive `with_logging`
- Outer function becomes the name of the decorator
- Must accept an instance of a function

```
1 def with_logging(func):
2     ...
3     pass
4
5
6 @with_logging
7 def greet():
8     print("Hello world!")
```

Decorator Implementation

- We want `with_logging` to return a function
- This is the function that will get called *instead* of what was actually called

```
1  def with_logging(func):
2      def wrapper():
3          pass
4
5      return wrapper
6
7  @with_logging
8  def greet():
9      print("Hello world!")
10
11 # calls wrapper() instead of greet()
12 greet()
```

Decorator Implementation

- We probably want our wrapper to call `func`
- But we can add additional behavior

```
 1 def with_logging(func):
 2     def wrapper():
 3         print("Begin code execution")
 4         func()
 5         print("End code execution")
 6
 7     return wrapper
 8
 9 @with_logging
10 def greet():
11     print("Hello world!")
12
13 # calls wrapper() instead of greet()
14 greet()
```

Decorator Arguments

- `greet` takes no parameters
- How could we fix this?
- Well, we could just copy the arguments of the function, into the decorator
- But that would limit our decorator to only being usable to that specific function
- Lisp didn't have this problem (`&body` + homoiconicity solved this)
- Let's see how Python handles this

*args

- Allows a function to have a variable number of arguments
- Wraps the data in a tuple (Basically an immutable list)

```
1 def add(*args):
2     sum = 0
3     for value in args:
4         sum += value
5 
6     return sum
7 
8 print(add(1, 2, 3))
9 print(add(1, 2, 3, 4, 5))
```

**kwargs

- Allows you to pass keyword arguments via =
- Wraps the data as a dictionary (map)
- Can mix this with `*args` if needed

```
def display_profile(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

display_profile(
    username="Reese Hatfield",
    email="reese.hatfield@wright.edu",
    uid="0101010101"
)
```

Decorator Arguments

- If we wanted to make our decorators truly reusable
- We need to ensure that it works
- Regardless of what arguments are passed
- Add `*args` and `**kwargs`
- Allow for any number of arguments and any number of keyword arguments

Decorator Arguments

- Wrapper should accept and pass to `func`
- May also want to capture the return value

```
1  def with_logging(func):
2      def wrapper(*args, **kwargs):
3          print("Begin code execution")
4          res = func(*args, **kwargs)
5          print("End code execution")
6          return res
7
8      return wrapper
9
10 @with_logging
11 def add(a, b):
12     return a + b
13
14 value = add(5, 6)
15 print(value)
```

Decorator Uses

- Logging is cool and all
- But decorators can be useful beyond simple logging
- Take a fibonacci function
- Write out what actually gets called
- *Memoization*: a programming optimization to cache already computer return values (of a pure function)

Memoization Decorator

- This code will slow to a crawl
- Think about how many times `fibonacci(3)` is *actually computed*
- Let's store it and look it up, instead of computing it, via a decorator

```
1 def fibonacci(value):
2     if value <= 1:
3         return 1
4
5     return fibonacci(value - 1) + fibonacci(value - 2)
6
7 for i in range(120):
8     print(fibonacci(i))
```

Memoization Decorator

- (I'm omitting `*args` + `**kwargs` here)

```
1  cache = {}
2
3  def memoizer(func):
4      global cache
5
6      def memoized_fn(value):
7          try:
8              result = cache[value]
9          except KeyError:
10             result = func(value)
11             cache[value] = result
12
13         return result
14
15     return memoized_fn
```

Decorators

- Decorators allow modification to functions via HOF
- Memoization or logging are not the only use of these
- Python also has many OOP decorators like:
- `@staticmethod`: Makes a method a static method
- `@classmethod`: Makes a method an instance method
- `@property`: Turn methods into getters

Java Annotations

- Likely familiar with a similar looking pattern in Java
- Java builds most of these in (`@Override`, `@FunctionalInterface`
- They are *fundamentally different* things
- Libraries like Lombok allow for similar behavior via `@Getter`+`@Setter`

```
1 @Override
2 public String toString(){
3     return "I am John Smith";
4 }
```

Questions?